Lab 08 — Persistent Storage — PV, PVC & StorageClass

ID: K8S-LAB-08 · **Category:** Core · **Level:** Engineer · **Duration:** 45 min

Focus: PersistentVolume, PersistentVolumeClaim, StorageClass, dynamic provisioning

Objective

Provision storage dynamically with a StorageClass, attach it to a Pod, and prove the data survives Pod deletion.

Concepts Covered

• The PV/PVC binding loop

• Reclaim policies (Retain vs Delete)

• Access modes (RWO/ROX/RWX)

Prerequisites

• A cluster with a default StorageClass (kind/minikube ship one; bare kubeadm needs local-path-provisioner)

Architecture

PVCs are the consumer-facing API. PVs are the supply. A StorageClass automates the supply by calling a CSI driver.

Lab Steps

01. Install local-path-provisioner (if no default class)

Gives the cluster a default StorageClass backed by local disk.

```
kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-provisioner/master/deploy/local-path-st
kubectl patch storageclass local-path -p '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-
```

Expected: storageclass.storage.k8s.io/local-path created.

Internals: The provisioner watches PVCs and creates a hostPath PV on demand.

02. Create a PVC and a Pod that uses it

PVC requests 1Gi. The provisioner creates a matching PV. Pod mounts it at /data.

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata: {name: data}
spec:
  accessModes: [ReadWriteOnce]
  resources: {requests: {storage: 1Gi}}
```

```
---
apiVersion: v1
kind: Pod
metadata: {name: writer}
spec:
  containers:
  - name: app
    image: busybox
    command: ["sh","-c","echo hello > /data/file; sleep 3600"]
    volumeMounts: [{mountPath: /data, name: vol}]
  volumes:
  - name: vol
    persistentVolumeClaim: {claimName: data}
EOF
```

Expected: PVC STATUS=Bound, Pod Running.

Internals: PV/PVC binding is atomic; once bound a PVC can only mount that PV.

03. Prove persistence

Deleting the Pod doesn't delete the PVC or its data.

```
kubectl exec writer -- cat /data/file
kubectl delete pod writer
kubectl apply -f - <<'EOF'
apiVersion: v1
kind: Pod
metadata: {name: reader}
spec:
  containers:
  - name: app
    image: busybox
    command: ["sh","-c","cat /data/file && sleep 3600"]
    volumeMounts: [{mountPath: /data, name: vol}]
  volumes:
  - name: vol
    persistentVolumeClaim: {claimName: data}
EOF
kubectl logs reader
```

Expected: `hello` printed by the second Pod.

Internals: Reclaim policy on dynamically-provisioned PVs defaults to Delete — but only triggers when the PVC itself is deleted.

Common Errors

PVC stuck Pending — Cause: No StorageClass / no PV matches · Fix: kubectl describe pvc to see the binder error

Pod can't mount volume — Cause: RWO PV already bound elsewhere, or node mismatch · Fix: Check PV claimRef and topology constraints

Troubleshooting

• `kubectl get pv,pvc -A`

• Check the provisioner pod's logs

Enterprise Relevance

Stateful workloads live or die by correct PVC sizing and reclaim policy. Wrong reclaim = data loss.

Validation

• Data outlives the Pod that wrote it

• You can name the three access modes

Related Incidents & Guide Sections

Incidents: K8S-INC-3004

• Storage / PV / PVC / CSI

• PVC Pending

Questions

Beginner

Q. What does a PVC do?

A. Requests storage matching a size + access mode.

Intermediate

Q. When does Reclaim=Delete actually delete?

A. When the PVC is deleted; deleting the Pod alone does not trigger it.

Advanced

Q. Why is RWX rare on block storage?

A. Block devices generally don't support multi-writer; you need a network filesystem.

Scenario

Q. PVC stuck Pending forever. What's the most likely cause?

A. No StorageClass or no provisioner — kubectl describe pvc shows the binder error.

Viva

Q. Why prefer StorageClass over manually creating PVs?

A. Dynamic provisioning — scales with demand, no manual ops.